Trabajo Práctico Integrador — Segunda Entrega (Unidades 4 a 8)

Food Store: Cierre Integral de la Capa de Datos (PostgreSQL 18 · MongoDB · SQLite 3.50+)
Estudiante: Fabrizio Simon · **Cátedra:** Base de Datos II (UTN FRM) · **Ciclo Lectivo:** 2026

Repositorios Oficiales en GitHub:
• **Parte 1 (Unidades 1 a 6):** https://github.com/simonfabrizio103/food-store-tp (Carpetas facultad/food-store-tp-unidad4/, u5/, u6/ en rama master, commit 202aecb).
• **Parte 2 (Unidades 7 y 8):** https://github.com/simonfabrizio103/food-store-tp-u7-u8 (Carpetas tp-unidad-7/ y tp-unidad-8/ en rama main, etiquetas oficiales **tp8** y **tpi-entrega2**).

Introducción y Enfoque Personal

Cuando me senté a encarar esta segunda entrega del Trabajo Práctico Integrador de Food Store, me quedó clarísimo que no se trataba de un trabajo práctico más donde tirás cuatro consultas para que devuelvan filas y pasás de largo. Acá el desafío era agarrar todo lo visto en la materia y hacer convivir un esquema que arrancó en un motor relacional tradicional como PostgreSQL, llevarlo al límite con optimización física, blindarlo con políticas de seguridad reales, someterlo a pruebas de desastre contra reloj, extenderlo hacia una base documental con MongoDB y, como frutilla del postre, resolver la operatoria de un food truck vendiendo en una feria sin señal usando SQLite.

A lo largo de estas semanas me di cuenta de que una base de datos en el mundo real no es un depósito estático de tablas: es un sistema vivo, sometido a bloqueos, cuellos de botella de CPU, caídas de tensión, ataques de inyección y fallas de conectividad. Si uno diseña pensando que la red nunca se cae o que los usuarios son de confianza, el sistema explota en el primer día de producción.

En este camino me apoyé en herramientas de IA (Kiro para las especificaciones EARS, OpenCode y LLMs para estructurar código rápido), pero con una regla innegociable: **la IA propone, pero el que firma la arquitectura y pone la cara soy yo**. Cada sentencia SQL, script de PowerShell y modelo documental lo leí, lo probé en mi máquina y, cuando la IA sugirió malas prácticas (como un VACUUM FULL en caliente o autonuméricos en dispositivos móviles desconectados), las descarté y corregí con fundamentos técnicos de ingeniería.

Unidad / Eje Temático	Motor	Problema Principal Resuelto	Resultado Clave Validado
Unidad 4 · Normalización	PostgreSQL	Violación de FNBC en lotes y saturación de CPU en dashboard.	Descomposición sin pérdida con vista de compatibilidad; 37.4% más rápido.
Unidad 5 · Seguridad	PostgreSQL	Mínimo privilegio, auditoría forense y fallo de login H-1.	Column-Level Security, H-1 corregido con DEFINER y anonimización PII.
Unidad 6 · Mantenimiento	PostgreSQL	Bloat de índices, backups automáticos y simulación de desastre.	RTO medido en 4.84 s y RPO de 1.70 min restaurando 200,001 órdenes.
Unidad 7 · NoSQL	MongoDB	Cuello de botella en lecturas web de pedidos y catálogo.	Modelado por agregados con líneas embebidas; 100% de paridad con PG.
Unidad 8 · Móvil	SQLite	Venta offline en feria sin señal y conciliación asíncrona.	Tablas STRICT, UUIDs y semáforo jerárquico; PG central invariable.

1. Unidad 4 · Normalización avanzada y redundancia controlada

1.1 El problema del esquema heredado y la violación de FNBC

Al analizar la tabla heredada control_lote_almacen, identifiqué que la clave primaria compuesta era (LoteID, ResponsableControlID). Sin embargo, existía la dependencia funcional no trivial:

ResponsableControlID → DepositoID

Como ResponsableControlID no era superclave de la tabla, violaba la Forma Normal de Boyce-Codd (FNBC). Esto provocaba redundancia masiva (el depósito se repetía por cada lote auditado) y anomalías graves: si un responsable cambiaba de depósito, un UPDATE fallido dejaba al auditor asignado a dos depósitos distintos simultáneamente.

1.2 Descomposición sin pérdida y Vista de Compatibilidad

Descompuse la relación en dos tablas normalizadas:
1. responsable_control (ResponsableControlID PK, DepositoID, NombreResponsable).
2. control_lote_almacen_nuevo (LoteID, ResponsableControlID FK, FechaControl, Estado, CantidadAuditada).

Para no romper las aplicaciones existentes ni obligar a reescribir reportes, implementé el patrón de **Vista de Compatibilidad**:

```
CREATE OR REPLACE VIEW control_lote_almacen AS
SELECT cla.LoteID, cla.ResponsableControlID, rc.DepositoID, cla.FechaControl, cla.Estado, cla.CantidadAuditada
FROM control_lote_almacen_nuevo cla JOIN responsable_control rc ON cla.ResponsableControlID = rc.ResponsableControlID;
```

De este modo, normalicé el almacenamiento físico sin que ningún sistema externo se viera afectado.

1.3 Desnormalización controlada para el Dashboard (categoria_venta_dia)

En el dashboard ejecutivo se consultaba constantemente el **Top 5 de categorías más vendidas por día**. Calcular SUM(cantidad * precio) agrupando sobre cientos de miles de filas en cada refresco de pantalla consumía **0.123 ms** por consulta con Seq Scan. Decidí desnormalizar creando la tabla agregada categoria_venta_dia con un índice B-tree compuesto (fecha, total_vendido DESC).

Salvaguardas de consistencia:

• **Trigger automático:** Creé un trigger AFTER INSERT OR UPDATE ON pedido_producto que actualiza los totales precalculados en tiempo real.
• **Auditoría periódica:** Implementé una consulta de control que compara la tabla desnormalizada contra la agregación pura. En mis pruebas reportó **0 filas desincronizadas**.
• **Rendimiento medido con EXPLAIN ANALYZE:** El tiempo de consulta cayó a **0.077 ms** (Bitmap Heap Scan), logrando una **mejora del 37.4% en CPU**.

2. Unidad 5 · Administración, seguridad y laboratorio forense

2.1 Arquitectura RBAC bajo el Principio de Mínimo Privilegio

Diseñé un esquema de control de acceso basado en roles (RBAC) en dos capas separando estrictamente responsabilidades:
• **Roles de grupo:** rol_admin_datos (administración total), rol_app_web (tienda online), rol_cajero (ventas físicas) y rol_auditor (auditoría).
• **Usuarios de conexión:** admin_datos y app_web asignados a sus respectivos grupos.
• **Column-Level Security:** La aplicación web tiene restringido el acceso: GRANT SELECT (id_usuario, nombre, activo) ON usuario TO rol_app_web;. Tiene terminantemente prohibido ver contraseñas o emails de clientes.

2.2 Auditoría nativa y Simulacro de Incidente

Activé en PostgreSQL log_connections = on, log_disconnections = on y log_statement = 'all'. Luego ejecuté un ataque simulado conectándome como app_web: intenté 4 veces invocar fn_autenticar con claves falsas, quise consultar emails ajenos (SELECT email FROM usuario), intenté elevar privilegios a admin_datos y crear usuarios hacker. PostgreSQL rechazó cada intento con permission denied, registrando la evidencia en los logs.

2.3 El Hallazgo Crítico H-1 y la Solución con SECURITY DEFINER

Al probar el login de clientes descubrí un bug crítico: fn_autenticar fue creada originalmente como SECURITY INVOKER. Como app_web carece de permiso de SELECT sobre la tabla usuario, PostgreSQL bloqueaba el login arrojando permission denied for table usuario. La aplicación no podía autenticar a nadie.

Remediación aplicada: En vez de cometer el error de darle SELECT a la app sobre toda la tabla, convertí la función a SECURITY DEFINER, le asigné el owner admin_datos y le blindé el esquema de búsqueda:

```
ALTER FUNCTION fn_autenticar(text, text) OWNER TO admin_datos;
ALTER FUNCTION fn_autenticar(text, text) SECURITY DEFINER;
ALTER FUNCTION fn_autenticar(text, text) SET search_path = public, pg_temp;
```

Con esto, la función corre con los privilegios de su dueño, verifica la contraseña y retorna solo un booleano, manteniendo a la app web aislada de los datos sensibles.

2.4 Hallazgo H-2 y Anonimización de Datos Personales (PII)

El log reveló que log_statement = 'all' guardaba contraseñas en texto claro. Para remediarlo creé anonimizar_log.py en Python para enmascarar credenciales e IPs antes de compartir trazas. Asimismo, creé la tabla usuario_anon con hash determinista y sal sobre los emails (md5(id || sal) || '@anon.example'), eliminando contraseñas pero conservando id_usuario. Las pruebas estadísticas de verificacion_equivalencia.md confirmaron **100% de paridad en agregados sin filtrar PII real**.

3. Unidad 6 · Mantenimiento, backup y simulacro de desastre

3.1 Mantenimiento preventivo en PostgreSQL (maintenance.sql)

En una base con 200,000 pedidos, el motor acumula miles de tuplas muertas por día debido al motor MVCC. Diseñé maintenance.sql con tres pilares:

• **VACUUM (ANALYZE, VERBOSE):** Limpia tuplas muertas y actualiza el mapa de espacio libre (FSM) sin bloquear lecturas ni escrituras concurrentes.
• **REINDEX TABLE CONCURRENTLY:** Desfragmenta y compacta los índices B-Tree inflados (bloat) construyendo un índice paralelo sin bloquear el acceso comercial.
• **Actualización de estadísticas:** Refresca las frecuencias en pg_statistic para que el planificador elija siempre index scans óptimos.
• **Salud del motor (health_check.sql):** Monitoreo de pg_statio_user_tables verificando un **Cache Hit Ratio superior al 99.2%**.

3.2 Estrategia de backups automatizados en PowerShell (backup_foodstore.ps1)

Programé un script en PowerShell para automatizar respaldos lógicos con nivel de producción:
• Valida que el servicio PostgreSQL esté en ejecución y comprueba que el disco posea al menos 500 MB libres antes de iniciar.
• Genera respaldos comprimidos con pg_dump.exe -F c -b -v (formato custom con compresión zlib, permitiendo restore selectivo y paralelo).
• Verifica que el archivo generado exista y tenga tamaño mayor a cero.
• Aplica rotación automática purgando backups con más de 7 días de antigüedad.

3.3 Simulacro de desastre contra reloj: Medición de RTO y RPO

Para validar nuestro plan de contingencia programé restore_test.ps1, simulando la caída total del servidor desde el backup foodstore_20261110_155502.dump (11.2 MB comprimido, 96 MB en disco, con 200,001 pedidos y 50,002 productos):

Métrica de Continuidad	Valor Medido Real	SLA Objetivo	Estado de Cumplimiento
RTO (Recovery Time Objective)	**4.84 segundos**	Menor a 15 minutos	**SUPERADO AMPLIAMENTE**
RPO (Recovery Point Objective)	**1.70 minutos**	Menor a 1 hora	**SUPERADO AMPLIAMENTE**
Integridad Estructural	0 claves foráneas rotas	100% de tablas y órdenes	**VALIDADO (validate_restore.sql)**

Esto demostró empíricamente que ante un incidente crítico, el sistema recupera su operatividad plena en menos de 5 segundos.

4. Unidad 7 · NoSQL y nuevas formas de almacenamiento (MongoDB)

4.1 Rompiendo el molde relacional: Diseño documental de pedidos

En la Unidad 7 comprendí que migrar a NoSQL no es volcar tablas relacionales en colecciones separadas para después juntarlas con $lookup. Diseñé la colección pedidos en MongoDB bajo el concepto de **Agregado (DDD)** con líneas y cliente embebidos:

```
{ _id: 2, fecha: ISODate("2026-11-10T12:00:00Z"), estado: "ENTREGADO",
  cliente: { id_cliente: 201, nombre: "Ana Gómez", email: "ana@example.com" },
  lineas: [
    { producto_id: 1, nombre: "Muzza", cantidad: 2, precio_unitario: 100.00 },
    { producto_id: 2, nombre: "Napolitana", cantidad: 1, precio_unitario: 120.00 }
  ], total: 320.00 }
```

Ventajas del modelo:
1. **Precio unitario congelado:** Si mañana la Muzza sube a $150 en la tabla maestra, la venta histórica de Ana sigue preservando los $100 originales.
2. **Lectura atómica en un único viaje a disco:** La app móvil consulta un pedido completo con findOne({ _id: 2 }) sin JOINs.

4.2 Consultas analíticas y Validación Cruzada con PostgreSQL

Implementé en consultas_mongodb.js 4 consultas complejas y sus contrapartes en consultas_postgresql.sql, ejecutándolas con el runner autónomo en memoria ejecutar_mongodb.js (usando mongodb-memory-server):
• **4.a (Pedidos entregados):** find({ estado: 'ENTREGADO' }) → Retornó pedidos 2, 4 y 7 en ambos motores.
• **4.b (Filtro en subdocumentos):** find({ 'lineas.producto_id': 3 }) → Retornó pedidos 2, 4 y 6 en ambos motores.
• **4.c (Total > $2000):** Retornó pedidos 4 ($2050) y 6 ($3500) en ambos motores.
• **4.d (Agrupación con $unwind y $group):** Ventas totales por producto.
Resultado: 100% de coincidencia exacta en montos y cantidades entre el motor documental y el relacional.

4.3 Decisión Arquitectónica: Enfoque Híbrido CQRS

No caí en el falso dilema 'relacional vs documental'. Adopté una arquitectura **CQRS (Command Query Responsibility Segregation)**:
• **PostgreSQL es el Core Transaccional (OLTP):** Única Fuente de Verdad (SSoT). Controla inventario físico, estados fiscales y auditoría.
• **MongoDB es el Read Model:** Almacena pedidos confirmados desnormalizados para responder consultas masivas desde apps web y móviles con tiempos de respuesta en milisegundos sin sobrecargar a PostgreSQL. Si MongoDB falla, se regenera desde el relacional.

5. Unidad 8 · Bases de datos móviles e integración (SQLite)

5.1 Operatoria del Food Truck en la feria: Venta Offline-First

El Food Truck vende en una feria gastronómica sin señal celular ni Internet. Diseñé la base SQLite (`schema_sqlite.sql`) bajo 4 reglas:
1. **PRAGMA foreign_keys = ON;:** Activación obligatoria de integridad referencial.
2. **Tablas STRICT:** Tipado estático forzado para impedir que el motor acepte strings en columnas de stock o precios.
3. **Precios en centavos enteros (INT):** Elimina errores acumulativos de redondeo de punto flotante de los tipos `REAL`.
4. **UUIDs v4 (TEXT PRIMARY KEY):** Si tres mozos toman pedidos en tres tablets desconectadas, usar autonuméricos generaría colisiones de IDs al reconectarse. Con UUID v4 cada pedido nace con un identificador unívoco a nivel mundial.

5.2 Algoritmo del Semáforo de Sincronización

En semaforo.sql implementé una consulta atómica con CTE y UPDATE ... FROM que evalúa las solicitudes contra el snapshot del servidor:
• **Rojo (RECHAZADO):** Si el producto no existe, fue dado de baja lógica (eliminado = 't'), está inactivo (disponible = 'f') o si la cantidad pedida supera el stock del servidor.
• **Amarillo (CONFLICTO):** Si hay stock y vigencia pero el precio del servidor cambió respecto al visto en la tablet. Permite renegociar con el cliente en mostrador.
• **Verde (SIMULADO_APROBADO):** Si precio, stock y disponibilidad coinciden al 100%.
Regla de Oro: El móvil propone, el servidor dispone. La tablet no altera stock ni confirma ventas fiscales.

5.3 Simulación de Casos C1 a C7 y Verificación en PostgreSQL

En simulador.py ejecuté la batería completa de pruebas sobre foodstore_movil.db:
• **C1 (Catálogo offline):** Consulta de productos con stock y activos sin conectarse a la red.
• **C7 (Integridad estricta):** Intentos de cantidad en 0 y producto inexistente abortados por CHECK y FOREIGN KEY.
• **C2 (Conflicto de precio):** Muzza sube de $100 a $125 → CONFLICTO; cliente acepta → reintento da SIMULADO_APROBADO.
• **C3 (Quiebre de stock):** Pedido de 50 Napolitanas con stock 10 en servidor → RECHAZADO.
• **C4 (Baja lógica / lápida):** Producto dado de baja → RECHAZADO; la fila física persiste en SQLite pero se oculta del catálogo activo.
• **C5 (Aprobación limpia):** Coincidencia total en precio y stock → SIMULADO_APROBADO.
• **C6 (Idempotencia):** Reenvío del mismo UUID rechazado por clave primaria; el semáforo no lo reevalúa.

Auditoría en PostgreSQL: Al finalizar las pruebas, verifiqué que total_pedidos continuó en **200,001** y el stock físico de los productos no varió ni una sola unidad. La regla de oro se cumplió.

6. Revisión Crítica de Inteligencia Artificial y los 5 Descartes Técnicos

Utilicé Kiro (especificaciones EARS) y OpenCode/LLMs para acelerar desarrollo, pero mantuve una auditoría humana rigurosa. A continuación expongo los cinco descartes técnicos fundamentales donde la intervención humana corrigió propuestas peligrosas de la IA:

ID / Unidad	Componente	Propuesta de la IA	Decisión Humana	Fundamentación Técnica y Motivo de Rechazo
CR-1 Unidad 6	maintenance.sql	Correr VACUUM FULL ANALYZE diariamente para compactar tablas.	**DESCARTADO DE PLANO**	VACUUM FULL solicita ACCESS EXCLUSIVE LOCK, bloqueando lecturas y escrituras durante minutos. En producción tiraría la tienda abajo. Mantuve VACUUM (ANALYZE, VERBOSE) concurrente.
CR-2 Unidad 8	schema_sqlite.sql	Usar claves autonuméricas INTEGER AUTOINCREMENT en pedidos.	**DESCARTADO DE PLANO**	En múltiples tablets desconectadas, los enteros autonuméricos colisionan inevitablemente al sincronizar en el servidor central. Se impuso **UUID v4**.
CR-3 Unidad 8	Triggers móviles	Triggers locales en SQLite para actualizar stock remoto en PG por socket.	**DESCARTADO DE PLANO**	Viola la premisa Offline-First: si no hay señal en la feria, la interfaz se colgaría esperando el timeout de red. La conciliación debe ser asíncrona mediante el semáforo.
CR-4 Unidad 7	Modelado NoSQL	Afirmar que 'MongoDB carece de soporte para transacciones ACID'.	**REFUTADO TÉCNICAMENTE**	MongoDB soporta transacciones ACID multi-documento desde v4.0, y en agregados con líneas embebidas la atomicidad es nativa a nivel de documento sin overhead de coordinación.
CR-5 Unidad 5	fn_autenticar	Conceder permisos directos de SELECT a app_web sobre usuario.	**DESCARTADO**	Viola el mínimo privilegio y expone hashes de contraseñas. Se resolvió convirtiendo la función a SECURITY DEFINER con search_path seguro.

Declaración de Uso de IA (DUIA): Todas las interacciones, especificaciones en Kiro, prompts empleados y fundamentaciones de descarte fueron asentadas en las bitácoras (BITACORA.md) y declaraciones juradas (DUIA.md) de ambos repositorios, acreditando el dominio conceptual y técnico del estudiante para defender cada línea de código en la instancia oral.

7. Conclusión Final y Aprendizajes de Carrera

Haber completado este proyecto integrador a lo largo de las 8 unidades cambió totalmente mi perspectiva sobre lo que significa trabajar con bases de datos. Cuando uno recién arranca a programar, piensa que una base de datos es tirar un CREATE TABLE, un par de INSERT y hacer andar una aplicación. Hoy me doy cuenta de que la base de datos es el corazón de cualquier negocio: si la base se bloquea, la empresa no factura; si pierde datos por un desastre y no tiene backups probados, la empresa quiebra; y si expone datos por una mala política de permisos, la empresa enfrenta problemas legales gravísimos.

Aprendí que optimizar no es agregar índices sin medir sino estudiar el plan del optimizador con EXPLAIN ANALYZE; que la seguridad radica en el mínimo privilegio real y funciones securizadas con SECURITY DEFINER; que un plan de contingencia requiere pruebas cronometradas de restauración (RTO de 4.84 s y RPO de 1.70 min); que en arquitecturas modernas la complementariedad entre PostgreSQL (núcleo transaccional), MongoDB (read-models desnormalizados) y SQLite (borde desconectado) permite construir soluciones empresariales de alta disponibilidad; y fundamentalmente, que el verdadero valor de un programador radica en tener el criterio técnico para auditar la Inteligencia Artificial y tomar las decisiones correctas.